

KeynetikPOS

Setting the Application Icon

A practical guide for Qt + CMake on Windows

Applies to Qt 6.11 / CMake / MinGW

Includes the specific repair required by KeynetikPOS

Verified against the repository on 19 July 2026

Contents

- 1 Four different icons — know which one you mean
- 2 Preparing the .ico file
- 3 Method A — CMake with a Windows resource script
- 4 Method B — qmake, for reference
- 5 The runtime window icon
- 6 The current state of KeynetikPOS
- 7 The repair, step by step
- 8 Verifying it worked
- 9 Troubleshooting

1 Four different icons — know which one you mean

Most confusion about Qt application icons comes from treating a single problem as four. They are set in different places, and setting one does nothing for the others.

Icon	Where it appears	How it is set
Executable icon	The .exe in Explorer, the taskbar, Alt+Tab, pinned shortcuts	Embedded into the binary by a Windows resource script (.rc) compiled into the target
Window icon	The title bar and, on some systems, the taskbar button	QApplication::setWindowIcon() at runtime, from a Qt resource
Installer icon	The Setup .exe itself and the wizard's own chrome	CPACK_NSIS_MUI_ICON / MUI_UNIICON in CMakeLists
Shortcut icon	Start Menu and desktop shortcuts	Inherited from the executable icon — nothing separate to set

The common trap

Setting the installer icon makes the *Setup* file look right while the installed application still shows a blank default. They are different icons. A correct installer icon is not evidence that the executable icon is working — which is precisely the situation this project is in (section 6).

1.1 What Qt does for you on Windows

On Windows, Qt's platform plugin falls back to the **executable's** embedded icon when no window icon has been set explicitly. This is why embedding an icon in the .rc often appears to fix the title bar and taskbar at the same time. It is still worth calling **setWindowIcon()** explicitly — it is predictable, it works cross-platform, and it covers dialogs that are created before the main window.

2 Preparing the .ico file

A Windows **.ico** is a container holding several bitmaps at different sizes. Windows picks the closest match for the context it is drawing.

2.1 Sizes to include

Size	Used for
16 × 16	Title bar, small Explorer list view
32 × 32	Desktop shortcuts, Alt+Tab
48 × 48	Explorer medium icons — the common default view
256 × 256	Large and extra-large thumbnails, the Properties dialog

A single-size .ico looks wrong everywhere except one size

If the file contains only a 256×256 image, Windows downscales it to 16×16 for the title bar with no hinting, and the result is a blurred smudge. If it contains only 48×48, the large thumbnail is upscaled and soft. Always ship at least the four sizes above in one file.

2.2 Producing a multi-size icon

From a square source image of 1024 × 1024 or larger, with ImageMagick:

```
magick logo.png -define icon:auto-resize=256,48,32,16 resources/app.ico
```

Any icon editor will do the same. What matters is that the four sizes are present in a single file. Verify the count before shipping:

```
# Byte 4-5 of an .ico header is the image count
$b = [byte[]]:new(6)
$fs = [IO.File]::OpenRead("resources\app.ico")
$null = $fs.Read($b, 0, 6); $fs.Close()
[BitConverter]::ToUInt16($b, 4)      # expect 4, not 1
```

Design note

A 16×16 rendering of a detailed logo is unreadable. Most well-made icon sets use a simplified mark at the small sizes and the full logo at 256. The .ico format supports exactly this — the bitmaps need not be mechanical downscales of one another.

3 Method A — CMake with a Windows resource script

This is the correct approach for a CMake project and the one KeynetikPOS is built around. A `.rc` file is listed among the target's sources; CMake compiles it with `windres` under MinGW, or `rc.exe` under MSVC, and links the result into the binary.

3.1 The minimal resource script

`resources/app.rc`:

```
#include <windows.h>

IDI_ICON1 ICON "app.ico"
```

Two rules that cause most failures

The identifier matters. Windows uses the icon with the lowest-sorting resource ID as the application icon. `IDI_ICON1` is the conventional choice — an arbitrary name may leave the icon embedded but unused.

The path is relative to the `.rc` file, not to the project root or the build directory. Use forward slashes, or escape backslashes as `\\`.

3.2 Wiring it into CMakeLists

```
set(WINDOWS_RC_FILE "")
if(WIN32)
    set(RC_SOURCE "${CMAKE_CURRENT_SOURCE_DIR}/resources/app.rc")
    if(EXISTS "${RC_SOURCE}")
        set(WINDOWS_RC_FILE "${RC_SOURCE}")
    endif()
endif()

qt_add_executable(MyApp
    ${PROJECT_SOURCES}
    ${WINDOWS_RC_FILE}    # <- the .rc must be in the source list
)
```

The guard means the same CMakeLists still configures on Linux and macOS, where the variable stays empty and contributes nothing.

3.3 Generating the script so the version stays in sync

Hardcoding a version string into the `.rc` guarantees it will drift from the real one. The better pattern is a template that CMake fills in. Name it `app.rc.in` and substitute at configure time:

```
# app.rc.in
IDI_ICON1 ICON "@CMAKE_CURRENT_SOURCE_DIR@/resources/app.ico"

VS_VERSION_INFO VERSIONINFO
FILEVERSION @PROJECT_VERSION_MAJOR@, @PROJECT_VERSION_MINOR@, @PROJECT_VERSION_PATCH@, 0
...
    VALUE "ProductVersion", "@APP_VERSION@"
```

```
# CMakeLists.txt
configure_file(
    "${CMAKE_CURRENT_SOURCE_DIR}/resources/app.rc.in"
    "${CMAKE_CURRENT_BINARY_DIR}/app.rc"
    @ONLY
)
set(WINDOWS_RC_FILE "${CMAKE_CURRENT_BINARY_DIR}/app.rc")
```

Use an absolute icon path in a generated .rc

The generated file lands in the **build** directory, so a relative path such as "**app.ico**" would be resolved against the build tree and fail. This is why the template above interpolates **@CMAKE_CURRENT_SOURCE_DIR@**.

4 Method B — qmake, for reference

If a project still uses qmake rather than CMake, no .rc file is needed for the icon alone. One line in the .pro file is enough:

```
RC_ICONS = resources/app.ico
```

qmake generates a resource script behind the scenes. Related variables populate the version block:

```
VERSION            = 2.1.0
QMAKE_TARGET_COMPANY = Keynetik Solutions
QMAKE_TARGET_PRODUCT = KeynetikPOS
QMAKE_TARGET_DESCRIPTION = Professional Point of Sale System
QMAKE_TARGET_COPYRIGHT = Copyright (C) 2026 Keynetik Solutions
```

Do not mix the two

If you supply your own **RC_FILE**, qmake stops generating one and **RC_ICONS** is ignored. Use one mechanism or the other.

This section is included for completeness. KeynetikPOS is a CMake project — use Method A.

5 The runtime window icon

The executable icon is embedded in the binary and is not readable as a file at runtime. For **setWindowIcon()** you need the image inside Qt's own resource system, which means a .qrc file and a raster format — typically PNG.

resources/resources.qrc

```
<RCC>
  <qresource prefix="/">
    <file>app.png</file>
  </qresource>
</RCC>
```

CMakeLists.txt

```
set(QT_RESOURCE_FILE "")
if(EXISTS "${CMAKE_CURRENT_SOURCE_DIR}/resources/resources.qrc")
  set(QT_RESOURCE_FILE "resources/resources.qrc")
endif()
# ... then include ${QT_RESOURCE_FILE} in the target sources
```

main.cpp — before any window is constructed

```
QApplication app(argc, argv);
app.setWindowIcon(QIcon(":/app.png"));
```

Set it on the application, not on each window

QApplication::setWindowIcon() applies to every top-level window including dialogs created later. Setting it per-window means every new dialog is a chance to forget. Call it once, immediately after constructing the application object and before the first window.

Use a reasonably large PNG — 256×256 — and let Qt scale it down. A 32×32 source will look poor on a high-DPI display.

6 The current state of KeynetikPOS

The pieces are half-assembled. The project has both a resource script and a correct template for generating one, but the two were never connected and the icon directive is disabled. Verified by reading the files on 19 July 2026.

Component	State
<code>resources/app.ico</code>	Present, but contains only one image. Should hold four sizes (section 2).
<code>resources/icon.ico</code>	Does not exist — yet this is the filename both resource scripts point at.
<code>resources/app.rc</code>	Compiled into the binary, but the IDI_ICON1 line is commented out. Supplies version information only.
<code>resources/app.rc.in</code>	A correct, complete template — but nothing calls configure_file on it, so it is dead code.
<code>CMakeLists.txt:31</code>	Points RC_SOURCE at <code>app.rc</code> , the static file, never at the template.
<code>resources/resources.qrc</code>	Does not exist. The CMake guard at line 38 therefore never fires.
<code>setWindowIcon</code>	Not called anywhere in the codebase.
<code>CMakeLists.txt:215</code>	The installer icon does correctly use <code>app.ico</code> — which is why <code>Setup.exe</code> looks right.

Consequences today

The built executable carries **no icon at all** — Explorer, the taskbar and Alt+Tab show the Windows default, and Start Menu and desktop shortcuts inherit that same blank icon because they point at the executable. Separately, the embedded version information reads **1.0.0.0** while the application is version **2.1.0**, because `app.rc` hardcodes it.

Only the installer looks correct, which is exactly the trap described in section 1: a right-looking `Setup.exe` conceals a blank application icon.

7 The repair, step by step

Ordered so that each step can be verified before the next. Steps 1–3 fix the executable icon and the version drift; steps 4–5 add the runtime window icon.

Step 1 — Produce a multi-size icon

Replace the single-image file with one containing all four sizes, keeping the existing name so the installer configuration at `CMakeLists.txt:215` continues to work:

```
magick logo.png -define icon:auto-resize=256,48,32,16 resources/app.ico
```

Step 2 — Point the template at the file that exists

`resources/app.rc.in` currently references `icon.ico`, which is not present. Change it to `app.ico`:

```
IDI_ICON1 ICON "@CMAKE_CURRENT_SOURCE_DIR@/resources/app.ico"
```

Step 3 — Generate the script instead of using the stale one

In `CMakeLists.txt`, replace the block at lines 29–35 so the template is configured and the generated file is used. This fixes the icon and the version drift in one change, because the template already interpolates the real version:

```
set(WINDOWS_RC_FILE "")
if(WIN32)
    configure_file(
        "${CMAKE_CURRENT_SOURCE_DIR}/resources/app.rc.in"
        "${CMAKE_CURRENT_BINARY_DIR}/app.rc"
        @ONLY
    )
    set(WINDOWS_RC_FILE "${CMAKE_CURRENT_BINARY_DIR}/app.rc")
endif()
```

`app.rc` then becomes redundant and should be deleted, so no one later edits the file that is no longer compiled.

Reconfigure, do not just rebuild

`configure_file` runs at CMake configure time. An incremental build may not pick up an edit to the template — re-run CMake, or simply build after touching `CMakeLists.txt`, which triggers a reconfigure.

Step 4 — Add a Qt resource for the window icon

Export the logo as `resources/app.png` at 256 × 256 and create `resources/resources.qrc` as shown in section 5. The existing guard at `CMakeLists.txt:38` detects the file automatically — no CMake change is needed for this step.

Step 5 — Set the icon at startup

In `main.cpp`, immediately after the `QApplication` is constructed and before any window is shown:

```
app.setWindowIcon(QIcon(":/app.png"));
```

Order matters here

This project shows a **LoginDialog** before **MainWindow** is constructed. Setting the icon on the application object before either is created ensures the login window is covered too — a per-window call on **MainWindow** would leave the first thing the user sees unbranded.

8 Verifying it worked

Check the executable itself, not a shortcut to it, and not the installer.

- 1 Rebuild, then open the build directory in Explorer and switch to **Large icons** view. The .exe should show the artwork.
- 2 Right-click the .exe → **Properties** → **Details**. File version and product version should now read **2.1.0**, not 1.0.0.0 — this confirms the generated script is the one being compiled.
- 3 Run the application. The title bar and taskbar button should both show the icon, including on the login dialog.
- 4 Repackage with cpack, install, and confirm the Start Menu and desktop shortcuts show the icon.

The version check can be scripted, and is the quickest confirmation that the **generated** resource script replaced the stale one:

```
$exe = "build\Desktop_Qt_6_11_0_MinGW_64_bit-Release\KeynetikPOS.exe"
[Diagnostics.FileVersionInfo]::GetVersionInfo($exe) |
    Select-Object FileVersion, ProductVersion, CompanyName
```

Version still 1.0.0.0 after rebuilding?

The old **app.rc** is still being compiled. Confirm it was deleted and that **WINDOWS_RC_FILE** points into the binary directory, then reconfigure.

9 Troubleshooting

Symptom	Cause and remedy
Icon unchanged in Explorer, correct elsewhere	Windows caches shell icons aggressively. Run ie4unit.exe -show , or restart Explorer. Copying the .exe to a new name will also show the true icon immediately.
No icon anywhere after a clean build	The .rc is not in the target's source list, or the ICON line is commented out, or the identifier is not IDI_ICON1 . Check all three.
windres reports it cannot open the icon	The path in the .rc is resolved relative to the .rc file. For a generated script in the build directory, use an absolute path via @CMAKE_CURRENT_SOURCE_DIR@ (section 3.3).
Title bar icon correct, taskbar wrong	Two different icons. The taskbar generally follows the executable resource, the title bar follows setWindowIcon . Set both.
Icon blurred at small sizes	The .ico contains only one large bitmap. Rebuild it with all four sizes (section 2).
Installer looks right, application does not	CPACK_NSIS_MUI_ICON is independent of the executable resource. Fix the .rc — see section 6.
Icon missing only on the login dialog	setWindowIcon was called on MainWindow rather than on the QApplication , after the dialog had already been shown.

Guide to setting the application icon in a Qt + CMake project, with the repair required by KeynetikPOS. Generated by docs/generate_app_icon_guide.py. Repository state in section 6 verified on 19 July 2026.